Hackathon Self-Serve Guide: Build an RL Environment, Train an LLM, Ship a Demo

0) What you are building

The core idea is not just to fine-tune a text model, but to build a **specialized LLM system** that can act inside an environment, get feedback, and improve through reinforcement learning. The practical stack discussed here is:

Environment → verifier/reward functions → TRL trainer → Unsloth for efficiency → deployment on OpenEnv / Spaces.

A strong project usually looks like one of these,

Please refer to [External] Apr '26 OpenEnv Hackathon Themes for theme guidelines on selecting & forming problem statements.

1) Start with the right project idea

Pick a task that has all three of these properties:

1. **The model can act step by step**
2. **You can verify success programmatically**
3. **The task is hard enough to be interesting, but not so hard that the model never succeeds**

This last point matters a lot. RL only works if the probability of getting a good answer is greater than zero. If your task is so hard that the model never gets any reward, you will burn compute and learn nothing.

Please refer to [External] Apr '26 OpenEnv Hackathon Themes for theme guidelines on selecting & forming problem statements.

A useful rule: **prefer tasks with crisp verification over tasks that only "look good" to a human.** RL gets easier when the reward is objective.

At a high level, your loop is:

1. Give the model a prompt
2. Let it generate an action, strategy, answer, or code
3. Execute that output in an environment or verifier
4. Convert the result into a reward
5. Update the model so higher-reward behavior becomes more likely

That is the practical mental model for RL here. The system samples many outputs, scores them, and shifts probability mass away from bad outputs and toward better ones.

One especially useful framing is that RL is like a more efficient version of repeated in-context improvement. Instead of repeatedly stuffing previous examples into the context, you let backpropagation store what worked into the weights.

3) Decide whether you need SFT first

Use this simple rule:

- If you have **a lot of good data**, use **SFT**
- If you **do not have data but can verify outputs**, use **RL**
- In many practical cases, do **a little SFT first**, then RL

Why this matters:

- SFT is generally more sample-efficient
- RL is useful when you can test outcomes but cannot cheaply author ideal traces
- RL often needs some warm start, formatting priming, or easy tasks first so that good rollouts happen at all

For hackathon teams, the best path is usually:

2. Add light formatting or task scaffolding if needed

3. Use RL for improvement, not as magic from scratch

4) Design the environment before you design the trainer

Treat the environment as a first-class artifact. It should define:

- **reset()**: start a fresh episode
- **step(action)**: apply an action and return the next result
- **state() / observation**: what the agent sees
- **reward**: what counts as progress or success

OpenEnv standardizes this so the same training code can work across many environments, instead of every team inventing a different API. That is one of the main reasons to use it in a hackathon.

Think about your environment in this order:

1. What does the agent observe?
2. What actions can it take?
3. What ends an episode?
4. How do you compute reward?
5. How do you stop abuse, infinite loops, or cheating?

5) Build the environment using OpenEnv

The intended workflow is to bootstrap an environment skeleton and then fill in the behavior. OpenEnv's CLI creates the scaffolding for you. The environment is implemented as a Python package and exposed via a FastAPI app.

Your implementation typically defines:

- action dataclass

- state representation
- environment methods like reset and step
- FastAPI wrapper / client-server interface

That gives you a clean separation:

- the **environment** handles world dynamics and scoring,
- the **trainer** handles optimization,
- and the **model** just learns to act inside the interface.

6) Keep the task simple at first

Do not begin with your hardest benchmark. Start with the easiest version of your environment that still proves the concept. This is where curriculum learning helps.

A good progression:

1. easy tasks with short horizons,
2. medium tasks with a little more branching,
3. harder tasks only after the model starts getting non-zero reward.

The principle is simple: **make success possible early**. If the model never sees successful trajectories, learning stalls.

7) Design rewards carefully

Your reward function is your task specification. If it is weak, incomplete, or easy to exploit, the model will optimize the wrong thing very efficiently.

A strong reward design usually includes multiple components, for example:

- execution success,
- correctness,

- timeouts,
- resource usage,
- safety constraints,
- and anti-cheating checks.

One explicit recommendation was to use **multiple independent reward functions**, not just one. If you only have a single reward signal, it is easier for the model to hack it. Multiple independent checks reduce that risk.

For example, for a coding environment:

- reward passing tests,
- penalize timeouts,
- reward format compliance,
- reject use of forbidden globals,
- and separately verify the function contract.

8) Protect yourself against reward hacking

Reward hacking is one of the biggest practical failure modes. The model may learn shortcuts that maximize your reward without solving the real task. Examples mentioned include:

- editing timers,
- caching results,
- abusing globals,
- mutating protected state,
- or exploiting environment bugs.

What to do:

2. Lock down execution where possible
3. Add time limits
4. Avoid unrestricted global state
5. Sample outputs frequently and inspect them
6. Terminate or roll back runs if behavior drifts badly

A particularly practical recommendation was to use a **locked-down function** or restricted execution approach so the model cannot rely on undeclared globals or hidden cached state.

Also, do not just let training run forever without checking generations. Periodic human inspection is still necessary.

9) Use process-aware feedback when you can

Naively assigning the same final reward to every token is inefficient. If possible, use richer supervision that distinguishes good intermediate steps from bad ones. That is the idea behind **process supervision**.

In practice, this can be approximated by:

- line-by-line checks,
- step-level verifiers,
- program trace analysis,
- or LLM-as-a-judge for intermediate reasoning.

But be careful: LLM-as-a-judge can itself be gamed. Use it as one signal, not the only signal.

For a hackathon, outcome-based verification plus a few lightweight process checks is usually the sweet spot.

10) Pick the right training stack

The intended stack here is:

- **Unsloth** to make RL training and inference more efficient
- **OpenEnv** to standardize environment interaction

This combination works because:

- OpenEnv gives you a common environment interface
- TRL gives you RL trainers like GRPO
- Unsloth reduces memory use and improves efficiency on top of TRL

One of the practical examples used the same prompt repeated many times, routed through an environment, with TRL driving training and Unsloth helping with performance.

11) Prefer GRPO / RLVR style training for verifiable tasks

The RL setup discussed here leans toward **RL with verifiable rewards**:

- instead of a learned reward model,
- use a verifier, test harness, regex check, executor, or environment.

GRPO was described as a more efficient evolution relative to older PPO-style setups, especially by simplifying away parts like the value model.

For hackathon purposes, the key practical takeaway is:

- if the task is verifiable,
- build the verifier first,
- then plug that verifier into RL training.

12) Keep inference fast

One important point: in RL for LLMs, **inference can dominate total runtime**. Over time, rollout generation often becomes the bottleneck, not the optimizer step.

That means your project speed depends heavily on:

- tight environment loops,
- low-overhead execution,
- and efficient model runtime.

This is one reason Unsloth matters in the stack, and another reason to avoid overly heavy environments early in the hackathon.

13) Deploy your environment early

OpenEnv environments are designed to be deployed as **Hugging Face Spaces**, which provide:

- a running server,
- a Git repository,
- and a container registry.

That gives you several ways to work:

- interact with the remote Space directly,
- install the client code from the repo,
- pull and run the container locally,
- or run the FastAPI app locally via Python/Uvicorn.

Why this is good for a hackathon:

- one shared source of truth,
- easier collaboration,
- easier demos,
- easier switching between local and remote execution.

14) Scale only after the environment is stable

There was a dedicated tutorial flow around:

1. environment,
2. deployment,
3. scaling,
4. training with TRL and Wordle.

Follow the same order.

Do **not** start with scale. First confirm:

- reset works,
- step works,
- rewards are sensible,
- timeouts work,
- logs are visible,
- and the environment can be run locally and remotely.

Only then:

- increase batch sizes,
- duplicate prompts or tasks,
- expand task diversity,
- and benchmark throughput.

Do not watch only one scalar. Monitor:

- overall reward,
- individual reward function columns,
- success indicators,
- timeout frequency,
- and generated strategies over time.

A very concrete suggestion was:

- watch whether the reward is going up,
- and separately watch critical columns like “function works.”

Also inspect actual generations during training. A rising reward is not enough if the model is learning to exploit bugs.

16) Save models correctly

If you use QLoRA / LoRA-style training, be careful when saving. One explicit warning was:

Do not upcast a 4-bit model to 16-bit and then merge the LoRA weights naively. That can badly damage model quality. Instead, use the proper merged-save path, or use the adapters directly.

For participants, that means:

- keep your training save path simple,
- test post-training inference immediately,
- and do not leave export until the end.

17) How to structure your team over the hackathon

A very effective team split is:

- builds reset/step/state
- adds timeouts and safety constraints
- makes local and remote execution work

Person B: Verifier / Rewards

- writes multiple reward functions
- adds anti-hacking checks
- makes failure cases visible

Person C: Training

- sets up TRL + Unsloth
- runs experiments
- tracks metrics and generations

Person D: Demo / Product

- prepares the Space demo
- creates a simple interface
- records examples and final benchmarks

This split matches the way the stack naturally decomposes in practice.

18) A practical 1-day execution plan

Phase 1: Pick a narrow task

Choose a small, verifiable environment. Avoid huge long-horizon tasks first.

Use OpenEnv init, implement reset/step/state, and get a local loop working.

Phase 3: Build rewards

Add at least 2–4 independent reward checks, plus timeout and anti-cheat logic.

Phase 4: Deploy

Push to a Space or run locally via container/Uvicorn so teammates can use the same environment.

Phase 5: Train small

Run a tiny TRL + Unsloth experiment first. Look at outputs, not just metrics.

Phase 6: Inspect for hacking

Sample generations. Check for globals, hacks, environment abuse, or suspicious shortcuts.

Phase 7: Add curriculum

If the model gets zero reward too often, simplify tasks or add easier start states.

Phase 8: Train bigger

Only after the loop is stable should you increase scale, batch size, or environment diversity.

Phase 9: Save and demo

Export the trained model correctly, test inference, and show before/after behavior.

19) What judges or reviewers will likely find compelling

The strongest hackathon projects usually show:

- a clear environment design,
- objective reward functions,
- evidence that the model improved,

- a reproducible deployment story,
- and a sharp demo.

A simple but strong demo format is:

1. baseline model attempt,
2. reward/verifier output,
3. trained model attempt,
4. measurable improvement,
5. short explanation of safeguards.

20) Suggested problem statement theme directions

Please Refer to [External] Apr '26 OpenEnv Hackathon Themes

21) Common mistakes to avoid

- Picking a task so hard that success probability is zero
- Using only one reward function
- Not checking for reward hacking
- Training before the environment is stable
- Relying only on average reward and not inspecting outputs
- Forgetting timeouts and sandbox limits
- Saving LoRA/QLoRA models incorrectly

22) Learning Resources

(Recommended) RL Environment Lecture Chapters:

[External] Meta OpenEnv Hackathon Participant Help Guide

Module 1: Why OpenEnv? (~7 min)
- Workshop 8:02–15:05 — https://www.youtube.com/watch?v=1jU05MlENOI&t=482s
- Sanyam: RL loop, fragmented env APIs, OpenEnv as universal interface, Gymnasium spec + Docker
- Alt: Mega Lecture 40:01–46:00 — https://www.youtube.com/watch?v=Jew4lhAiqnw&t=2401s

Module 2: Using Existing Envs (~7.5 min)
- Workshop 35:33–43:05 — https://www.youtube.com/watch?v=1jU05MlENOI&t=2133s
- Ben: Hub org, env collections, 3 Space interfaces (server/repo/registry), `from_hub`
- Alt: Mega Lecture 1:24:11–1:30:00 — https://www.youtube.com/watch?v=Jew4lhAiqnw&t=5051s

Module 3: Deploying Envs (~9 min)
- Mega Lecture 1:30:00–1:39:07 — https://www.youtube.com/watch?v=Jew4lhAiqnw&t=5400s
- Ben: live `openenv init`, scaffold, running locally, `openenv push`, Docker run from Space
- Alt: Workshop 43:05–48:30 — https://www.youtube.com/watch?v=1jU05MlENOI&t=2585s

Module 4: Building Your Own (~6.5 min)
- Workshop 43:45–50:20 — https://www.youtube.com/watch?v=1jU05MlENOI&t=2625s
- Ben: scaffold files, business logic (reset/step), models, client, publishing
- Alt: Mega Lecture 1:33:30–1:39:07 — https://www.youtube.com/watch?v=Jew4lhAiqnw&t=5610s

Module 5: Training + TRL (~14 min)
- Mega Lecture 1:53:20–2:07:12 — https://www.youtube.com/watch?v=Jew4lhAiqnw&t=6800s
- Lewis: Wordle GRPO walkthrough — rollout function, reward shaping, GRPOTrainer, live training
- Alt: Workshop 22:24–34:12 — https://www.youtube.com/watch?v=1jU05MlENOI&t=1344s